TEMPLE COLLEGE
WORKFORCE DEVELOPMENT

CHIEFOS

STUDENT REVIEW PACKET

Week 9

Compose + Postgres, SQLAlchemy, error handling — Module 03 checkpoint

Phase 2 — Building Real Apps · Weeks 7–12 · Modules 03–04

No D2L login required. This is the same lesson text posted in Brightspace for this week. Work the lessons in order on your own machine. Keep each checkpoint screenshot or pasted output. Bring it to the live session, or email it to your instructor, instead of submitting in D2L. Module quizzes stay in D2L and are not included here.

In this packet

1. 03.09 Docker Compose and Postgres
2. 03.10 Database with SQLAlchemy
3. 03.11 Error Handling and Status Codes
4. 03.12 Final Checkpoint

Learning objective: Orchestrate the FastAPI app and a Postgres database with `docker-compose.yml`, using a healthcheck, a named volume, a bind-mounted hot-reload dev loop, and a `.env` file for secrets.

Lesson 09: Docker Compose and Postgres

By the end, you will have a `docker-compose.yml` defining two services — your FastAPI app and a Postgres database — running side by side, with hot-reload mounted in.

~40 minutes of typing (this lesson runs long — it's normal)

> Compose is YAML that orchestrates multiple containers. You declare services, volumes, networks; one `docker compose up` and Compose does the wiring.

---

1. Make sure you're in the project directory and any leftover container from lesson 08 is removed.

```
cd ~/docker-course/fastapi-app
docker ps -a --filter name=fastapi-app -q | xargs -r docker rm -f
```

2. Create `docker-compose.yml`.

```
touch docker-compose.yml
```

3. Open it in VS Code.

4. Type the top-level structure. Compose YAML is indentation-sensitive (2 spaces, no tabs).

```
services:
  api:
    build: .
    ports:
      - "8000:8000"
    volumes:
      - ./:/app
    environment:
      DATABASE_URL: postgresql://postgres:devpass@db:5432/appdb
    depends_on:
      db:
        condition: service_healthy
    command: uvicorn main:app --host 0.0.0.0 --port 8000 --reload

  db:
    image: postgres:16
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: devpass
      POSTGRES_DB: appdb
    volumes:
      - pgdata:/var/lib/postgresql/data
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 2s
      timeout: 3s
      retries: 10

volumes:
  pgdata:
```

5. Save. Read it back.

```
cat docker-compose.yml
```

6. Read each block aloud before starting:

- `api` — built from your Dockerfile, port 8000 mapped, code mounted as a bind volume, env var pointing at the db service, waits for db to be healthy, runs uvicorn with reload.
- `db` — official postgres image, password baked in for dev, data persisted in a named volume `pgdata`, port 5432 mapped (for tools), healthcheck so api waits.
- `volumes:` — declares the named volume Postgres uses.

7. Bring up the stack. The `-d` runs detached; without it the logs of both containers stream together.

```
docker compose up -d
```

You'll see `Creating network ...`, `Creating volume ...`, `Creating fastapi-app-db-1`, `Creating fastapi-app-api-1`.

8. List the services.

```
docker compose ps
```

You should see two services, both `Up`. The `db` service should show `(healthy)` after a few seconds.

9. Tail the api logs.

```
docker compose logs api
```

You should see uvicorn's startup banner.

10. Hit the api.

```
curl http://localhost:8000
```

```
{"hello":"fastapi"}
```

11. Hit `/items`.

```
curl http://localhost:8000/items
```

```
[]
```

12. Now feel the **bind-mount hot reload**. Edit `main.py`, change `read_root` to:

```
    return {"hello": "from-compose"}
```

13. Save. Look at the api logs — uvicorn should reload because of the `--reload` flag and the volume mount.

```
docker compose logs api --tail=10
```

You should see `WatchFiles detected changes in 'main.py'. Reloading...`.

14. Curl.

```
curl http://localhost:8000
```

```
{"hello":"from-compose"}
```

(Same dopamine loop as the venv version, now in Docker.)

15. Revert the change to `{"hello": "fastapi"}` and save.

16. Now poke at Postgres directly. Get a shell into the db container.

```
docker compose exec db psql -U postgres -d appdb
```

You should see the Postgres prompt:

```
psql (16.x)
appdb=#
```

17. List databases.

```
\l
```

You should see `appdb` in the list.

18. List tables — should be empty.

```
\dt
```

```
Did not find any relations.
```

19. Run a trivial query.

```
SELECT version();
```

20. Exit psql.

```
\q
```

21. Confirm the data volume exists separately from the container.

```
docker volume ls | grep pgdata
```

You should see your `fastapi-app_pgdata` volume.

22. Restart the db service — your data (well, the empty database) survives.

```
docker compose restart db
docker compose exec db psql -U postgres -d appdb -c "SELECT 1;"
```

```
 ?column?
----------
        1
```

23. Stop the stack.

```
docker compose down
```

```
Stopping fastapi-app-api-1 ... done
Stopping fastapi-app-db-1  ... done
Removing fastapi-app-api-1 ... done
Removing fastapi-app-db-1  ... done
Removing network fastapi-app_default
```

Note: `down` does **not** remove the named volume `pgdata`. To wipe data, use `docker compose down -v`.

24. Bring it back up.

```
docker compose up -d
```

25. Confirm the api is back up.

```
curl http://localhost:8000
```

```
{"hello":"fastapi"}
```

26. Add a `.env` file to keep the password out of the YAML. First, edit `docker-compose.yml` and replace the two `devpass` strings with `${POSTGRES_PASSWORD}` and the URL with `${DATABASE_URL}`. Add at the top of each service's environment list as needed. The relevant changes:

In the `api` service's `environment`:

```
      DATABASE_URL: ${DATABASE_URL}
```

In the `db` service's `environment`:

```
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
```

27. Create `.env` (Compose reads this automatically).

```
cat > .env <<'EOF'
POSTGRES_PASSWORD=devpass
DATABASE_URL=postgresql://postgres:devpass@db:5432/appdb
EOF
```

28. Add `.env` to `.gitignore` so secrets don't get committed.

```
echo ".env" >> .gitignore
```

29. Recreate the stack to pick up the env file.

```
docker compose up -d --force-recreate
```

30. Confirm everything still works.

```
curl http://localhost:8000
docker compose exec db psql -U postgres -d appdb -c "SELECT 1;"
```

31. Log it.

```
echo "Compose: api + postgres up, hot-reload via bind mount" >> ~/docker-
course/notes/notes.txt
```

Watch out

- **YAML indentation.** 2 spaces, never tabs. A misaligned key kills Compose with a cryptic error.
- **The hostname is `db`, not `localhost`.** From inside the api container, the postgres host is the **service name** in compose. Connection string: `postgresql://postgres:devpass@db:5432/appdb`. Beginners type `localhost` and spend 30 minutes debugging.
- **`depends_on: condition: service_healthy`** — this is what makes api wait until db is actually accepting connections, not just started. Without the healthcheck, api starts before postgres is ready and

crashes on the first DB query.
- `docker compose down` keeps the volume. `docker compose down -v` wipes it (deletes all data). Be careful.
- The `--reload` flag plus the bind mount (`./:/app`) is what gives you hot-reload inside the container. If you leave either out, you lose the dev loop.

Checkpoint

```
docker compose ps
```

Should show two services running, db `(healthy)`.

```
curl -s http://localhost:8000
```

Should print `{"hello":"fastapi"}`.

```
docker compose exec db psql -U postgres -d appdb -c "SELECT 1;"
```

Should print `1`.

```
docker compose logs api --tail=5
```

Should show recent uvicorn output.

```
docker volume ls | grep pgdata
```

Should show your named volume.

Next up: 10-database-with-sqlalchemy.md — Connect FastAPI to Postgres with SQLAlchemy

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Replace the in-memory item store with Postgres persistence using a SQLAlchemy engine, ORM models, and a `get_db` session dependency, then verify the rows directly in psql.

Lesson 10: Database with SQLAlchemy

By the end, the in-memory `fake_items_db` is gone. Items are read from and written to Postgres via SQLAlchemy. You'll see your rows in `psql` after POSTing them.

~45 minutes of typing (this lesson runs long)

> SQLAlchemy is the most popular Python ORM. We'll use the SQLAlchemy 2.0 style: an `engine`, a `SessionLocal`, a `Base` class for ORM models, and FastAPI dependencies for getting a session per request.

---

1. Make sure Compose is up and the venv is active in your local terminal (we'll edit files locally; Compose's bind mount picks them up).

```
cd ~/docker-course/fastapi-app
docker compose ps
source .venv/bin/activate
```

2. Add SQLAlchemy and the Postgres driver to `requirements.txt` so they're installed in the image.

```
echo "sqlalchemy==2.0.30" >> requirements.txt
echo "psycopg2-binary==2.9.9" >> requirements.txt
```

3. Confirm.

```
tail -n 5 requirements.txt
```

You should see the two new lines.

4. Rebuild the api image (Compose will use the cached layers for everything except pip install).

```
docker compose build api
```

5. Restart the api service.

```
docker compose up -d
```

6. Confirm the api is healthy.

```
curl http://localhost:8000
```

```
{"hello":"fastapi"}
```

7. Create `database.py` at the project root.

```
touch database.py
```

8. Open `database.py` in VS Code. Type:

```
import os

from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker


DATABASE_URL = os.environ["DATABASE_URL"]

engine = create_engine(DATABASE_URL, echo=False, future=True)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine,
future=True)
Base = declarative_base()


def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

9. Save.

10. Read each block:

- `DATABASE_URL` is read from the env (Compose already sets it for the api service).
- `engine` is the connection pool to Postgres.
- `SessionLocal` is a factory for sessions — one per request.

- `Base` is what ORM models inherit from.
- `get_db()` is a generator-style FastAPI dependency: it opens a session, yields it to the route, closes it after.

11. Create `models.py` at the project root.

```
touch models.py
```

12. Open `models.py`. Type:

```
from sqlalchemy import Column, Integer, String, Float, Boolean

from database import Base


class Item(Base):
    __tablename__ = "items"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, nullable=False)
    price = Column(Float, nullable=False)
    is_offer = Column(Boolean, nullable=True)
```

13. Save.

14. Now wire model creation at startup. Open `main.py` and add to the imports:

```
from database import Base, engine
import models  # noqa: F401  -- registers Item with Base.metadata
```

15. Below `app = FastAPI()`, add the table-creation call:

```



Base.metadata.create_all(bind=engine)
```

16. Save. Watch `docker compose logs api --tail=20` — you should see SQLAlchemy log a `CREATE TABLE items (...)` and uvicorn reload cleanly.

17. Open `psql` and confirm the table exists.

```
docker compose exec db psql -U postgres -d appdb -c "\dt"
```

```
 Schema | Name  | Type  |  Owner
--------+-------+-------+----------
 public | items | table | postgres
```

18. Now rewrite the items router to use the database. Open `routers/items.py`.

19. Replace the entire file with:

```python
from fastapi import APIRouter, Depends
from pydantic import BaseModel
from sqlalchemy.orm import Session

from database import get_db
from dependencies import get_current_user
import models


router = APIRouter(prefix="/items", tags=["items"])


class ItemIn(BaseModel):
    name: str
    price: float
    is_offer: bool | None = None


class ItemOut(ItemIn):
    id: int

    model_config = {"from_attributes": True}


@router.get("", response_model=list[ItemOut])
def list_items(db: Session = Depends(get_db)):
    return db.query(models.Item).all()


@router.get("/{item_id}", response_model=ItemOut)
def read_item(item_id: int, db: Session = Depends(get_db)):
    item = db.query(models.Item).filter(models.Item.id == item_id).first()
    return item


@router.post("", response_model=ItemOut)
def create_item(item: ItemIn, db: Session = Depends(get_db)):
    db_item = models.Item(**item.model_dump())
    db.add(db_item)
    db.commit()
    db.refresh(db_item)
    return db_item
```

```
@router.get("/me/protected")
def protected(user: dict = Depends(get_current_user)):
    return {"message": f"Hello {user['username']}, you reached a protected route"}
```

20. Save. Watch the api reload.

21. Read what changed:

- **Two Pydantic models now**: `ItemIn` for incoming requests (no `id`), `ItemOut` for responses (includes `id`).
- `model_config = {"from_attributes": True}` lets Pydantic build `ItemOut` from a SQLAlchemy ORM object.
- Each route takes a `db: Session = Depends(get_db)` — a fresh session per request, auto-closed.
- `db.add()` + `db.commit()` + `db.refresh()` is the standard SQLAlchemy create pattern.

22. POST an item.

```
curl -X POST http://localhost:8000/items \
  -H "Content-Type: application/json" \
  -d '{"name": "Real Apple", "price": 0.50}'
```

```
{"name":"Real Apple","price":0.5,"is_offer":null,"id":1}
```

Notice the `id` is back from the database.

23. POST another.

```
curl -X POST http://localhost:8000/items \
  -H "Content-Type: application/json" \
  -d '{"name": "Real Bread", "price": 2.50, "is_offer": true}'
```

24. List them.

```
curl http://localhost:8000/items
```

```
[{"name":"Real Apple","price":0.5,"is_offer":null,"id":1},{"name":"Real
Bread","price":2.5,"is_offer":true,"id":2}]
```

25. Read one by ID.

```
curl http://localhost:8000/items/1
```

```
{"name":"Real Apple","price":0.5,"is_offer":null,"id":1}
```

26. Now look in the actual database.

```
docker compose exec db psql -U postgres -d appdb -c "SELECT * FROM items;"
```

```
 id |    name    | price | is_offer
----+------------+-------+----------
  1 | Real Apple |   0.5 |
  2 | Real Bread |   2.5 | t
```

You're seeing your rows in Postgres. Real persistence.

27. Restart the stack to prove the data sticks.

```
docker compose restart api
```

28. List items again.

```
curl http://localhost:8000/items
```

The two items are still there.

29. Bring everything down — but **keep the volume**.

```
docker compose down
```

30. Bring it back up.

```
docker compose up -d
```

31. List items.

```
sleep 2 && curl http://localhost:8000/items
```

The two items are still there. The named volume preserved the data.

32. Update the users router to also use the DB. Open `routers/users.py` and replace it with:

```
from fastapi import APIRouter, Depends
from pydantic import BaseModel
from sqlalchemy import Column, Integer, String
from sqlalchemy.orm import Session

from database import Base, get_db
from dependencies import get_current_user


class UserModel(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, nullable=False)
    email = Column(String, nullable=False, unique=True)


router = APIRouter(
    prefix="/users",
    tags=["users"],
    dependencies=[Depends(get_current_user)],
)


class UserIn(BaseModel):
    name: str
    email: str


class UserOut(UserIn):
    id: int

    model_config = {"from_attributes": True}


@router.get("", response_model=list[UserOut])
def list_users(db: Session = Depends(get_db)):
    return db.query(UserModel).all()


@router.post("", response_model=UserOut)
def create_user(user: UserIn, db: Session = Depends(get_db)):
    db_user = UserModel(**user.model_dump())
    db.add(db_user)
```

```
    db.commit()
    db.refresh(db_user)
    return db_user
```

33. Save. Reload happens automatically.

34. The new `users` table doesn't exist yet — `Base.metadata.create_all` ran at startup before the new model was registered. Restart the api to re-run startup.

```
docker compose restart api
```

35. Confirm the table.

```
docker compose exec db psql -U postgres -d appdb -c "\dt"
```

You should see both `items` and `users`.

36. POST a user.

```
curl -X POST http://localhost:8000/users \
  -H "Content-Type: application/json" \
  -H "X-Token: secret-token" \
  -d '{"name": "Ada", "email": "ada@example.com"}'
```

```
{"name":"Ada","email":"ada@example.com","id":1}
```

37. List users.

```
curl http://localhost:8000/users -H "X-Token: secret-token"
```

```
[{"name":"Ada","email":"ada@example.com","id":1}]
```

38. Confirm in psql.

```
docker compose exec db psql -U postgres -d appdb -c "SELECT * FROM users;"
```

39. Log it.

```
echo "SQLAlchemy: items + users persisted in postgres" >> ~/docker-
course/notes/notes.txt
```

Watch out

- **`models` import side-effects.** The `import models` line in `main.py` is what registers the `Item` table with `Base.metadata`. Without it, `create_all` won't create the table. The `# noqa` comment is a hint to linters that the import is intentional even though `models` isn't directly referenced.
- **`from_attributes` (formerly `orm_mode`)** is the Pydantic v2 setting that lets a Pydantic model serialize from a SQLAlchemy object. Without it, FastAPI returns the raw object and serialization fails.
- **Adding a new `Base`-derived model after startup** doesn't auto-create the table. Restart the api (`docker compose restart api`) to re-run `create_all`.
- **`db.refresh(db_item)`** after commit is what loads the auto-generated `id` back from the database. Skip it and your response has `id: null`.
- For real apps, use **Alembic** for schema migrations rather than `create_all`. We're keeping it simple.

Checkpoint

```
docker compose exec db psql -U postgres -d appdb -c "\dt"
```

Should show `items` and `users`.

```
curl -s -X POST http://localhost:8000/items -H "Content-Type: application/json" -d
'{"name": "Test", "price": 1}' | python -m json.tool
```

Should return JSON including a numeric `id`.

```
docker compose exec db psql -U postgres -d appdb -c "SELECT count(*) FROM items;"
```

Should print a non-zero count.

```
curl -s http://localhost:8000/items | python -m json.tool
```

Should list every item you've POSTed.

Next up: 11-error-handling-and-status-codes.md — Return correct HTTP status codes

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Return correct HTTP status codes (201, 204, 400, 404) by setting `status_code` on route decorators and raising `HTTPException` for missing resources and business-rule violations.

Lesson 11: Error Handling and Status Codes

By the end, your API returns 201 on POST, 404 when an item doesn't exist, and 204 on successful DELETE — all using `HTTPException` and FastAPI's `status_code` argument.

~25 minutes of typing

> Default status codes are 200 for everything. That's wrong. POST should be 201 (Created), DELETE should be 204 (No Content), missing resources should be 404 (Not Found).

---

1. Compose should still be up from lesson 10. Confirm.

```
cd ~/docker-course/fastapi-app
docker compose ps
```

2. Open `routers/items.py`.

3. Add `HTTPException` and `status` to the FastAPI import. Update the line:

```
from fastapi import APIRouter, Depends, HTTPException, status
```

4. Update the `read_item` route to return 404 when the item isn't found. Replace the existing `read_item` with:

```
@router.get("/{item_id}", response_model=ItemOut)
def read_item(item_id: int, db: Session = Depends(get_db)):
    item = db.query(models.Item).filter(models.Item.id == item_id).first()
    if item is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Item not
found")
    return item
```

5. Save.

6. Curl a non-existent item.

```
curl -i http://localhost:8000/items/999999
```

```
HTTP/1.1 404 Not Found
...
{"detail":"Item not found"}
```

7. Curl an existing item (use an id you POSTed earlier).

```
curl -i http://localhost:8000/items/1
```

```
HTTP/1.1 200 OK
...
{"name":"Real Apple",...}
```

8. Now make POST return 201. Update the `create_item` decorator:

```
@router.post("", response_model=ItemOut, status_code=status.HTTP_201_CREATED)
def create_item(item: ItemIn, db: Session = Depends(get_db)):
    db_item = models.Item(**item.model_dump())
    db.add(db_item)
    db.commit()
    db.refresh(db_item)
    return db_item
```

9. Save.

10. Curl a POST and check the status.

```
curl -i -X POST http://localhost:8000/items \
  -H "Content-Type: application/json" \
  -d '{"name": "ErrCheck", "price": 1.0}'
```

```
HTTP/1.1 201 Created
...
{"name":"ErrCheck","price":1.0,"is_offer":null,"id":...}
```

11. Now add a DELETE route. Append to `routers/items.py`:

```
@router.delete("/{item_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_item(item_id: int, db: Session = Depends(get_db)):
    item = db.query(models.Item).filter(models.Item.id == item_id).first()
    if item is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Item not
found")
    db.delete(item)
    db.commit()
    return None
```

12. Save.

13. Pick an existing item id (e.g. 1) and delete it.

```
curl -i -X DELETE http://localhost:8000/items/1
```

```
HTTP/1.1 204 No Content
...
```

(204 means success with no response body — the standard for successful deletes.)

14. Try to read it now.

```
curl -i http://localhost:8000/items/1
```

```
HTTP/1.1 404 Not Found
...
{"detail":"Item not found"}
```

15. Try to delete the same item again.

```
curl -i -X DELETE http://localhost:8000/items/1
```

```
HTTP/1.1 404 Not Found
{"detail":"Item not found"}
```

16. Confirm in the DB.

```
docker compose exec db psql -U postgres -d appdb -c "SELECT * FROM items WHERE
id=1;"
```

```
 id | name | price | is_offer
----+------+-------+----------
(0 rows)
```

17. Add a custom error class. Sometimes you want a body other than `{"detail": "..."}`. Add a 400 case for invalid prices. Update the `create_item` route's body to validate (right after `def create_item(...)`):

```
    if item.price <= 0:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="price must be positive",
        )
```

18. Save.

19. Try a negative-priced item.

```
curl -i -X POST http://localhost:8000/items \
  -H "Content-Type: application/json" \
  -d '{"name": "Bad", "price": -1}'
```

```
HTTP/1.1 400 Bad Request
{"detail":"price must be positive"}
```

20. And zero.

```
curl -i -X POST http://localhost:8000/items \
  -H "Content-Type: application/json" \
  -d '{"name": "Zero", "price": 0}'
```

```
HTTP/1.1 400 Bad Request
```

21. A valid one still works.

```
curl -i -X POST http://localhost:8000/items \
  -H "Content-Type: application/json" \
  -d '{"name": "Good", "price": 5}'
```

```
HTTP/1.1 201 Created
```

22. Open `/docs`. Refresh. Each route now shows its proper response codes (201, 204, 404, 400).

23. Try the DELETE from Swagger UI. Find an item id in the list, paste it into the DELETE form, click Execute. The response body is empty and the status is 204 — Swagger shows both.

24. Log it.

```
echo "Status codes: 201/204/400/404 wired in" >> ~/docker-course/notes/notes.txt
```

Status code quick reference

Code	Meaning	When to use
200	OK	Default for GET; POST/PUT that return the resource
201	Created	POST that creates a new resource
204	No Content	Successful DELETE; PUT that doesn't return the resource
400	Bad Request	Caller's input is wrong (business rule violation)
401	Unauthorized	Caller didn't authenticate
403	Forbidden	Caller authenticated but lacks permission
404	Not Found	Resource doesn't exist
409	Conflict	Business state prevents the action (e.g. duplicate email)
422	Unprocessable Entity	Pydantic validation rejected the body — FastAPI does this automatically
500	Internal Server Error	Your code crashed — FastAPI does this automatically when an exception escapes

---

Watch out

- `status_code=status.HTTP_201_CREATED` is on the **decorator**, not on the return statement. The decorator sets the success status; `HTTPException` overrides it for errors.
- **204 must not return a body.** FastAPI honors this — even if you `return {"foo": "bar"}` from a 204 route, the body is dropped. Just `return None`.
- `HTTPException(detail=...)` — `detail` is what shows up in the `{"detail": "..."}` response. Use clear, user-actionable messages, but don't leak internal details (no stack traces).
- **Validation errors are always 422**, not 400. 400 is for *your* business rules; 422 is for Pydantic's structural validation.

Checkpoint

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8000/items/999999
```

Should print `404`.

```
curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:8000/items -H
"Content-Type: application/json" -d '{"name":"X","price":1}'
```

Should print `201`.

```
curl -s -o /dev/null -w "%{http_code}\n" -X DELETE
http://localhost:8000/items/$(curl -s http://localhost:8000/items | python -c
"import sys,json;print(json.load(sys.stdin)[-1]['id'])")
```

Should print `204` (deletes the most-recently created item).

```
curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:8000/items -H
"Content-Type: application/json" -d '{"name":"X","price":-1}'
```

Should print `400`.

Next up: 12-final-checkpoint.md — Build a `User` resource end-to-end from memory

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Build a complete `Profile` resource from memory — SQLAlchemy model, router, protected DELETE, and correct status codes — and pass a 10-step curl acceptance test.

Lesson 12: Final Checkpoint

Time: ~30 min — **no instructor help, no peeking at solutions** until you've tried.

This is the cumulative test. You'll add a real, working resource to your app from memory using only the patterns from this module: venv, FastAPI, Pydantic, routers, dependencies, Docker Compose, Postgres, SQLAlchemy, and proper status codes.

If you get stuck on a step, reread the relevant earlier lesson — but **don't** open `solutions/12-solution.md` until you've finished or genuinely can't move forward.

The spec

Extend the existing app with a fully-functional `Profile` resource. Treat this like a real backend ticket.

Model: `profiles` table with these columns:

- `id` — int, primary key, auto-increment
- `email` — string, required, unique
- `display_name` — string, required
- `created_at` — timestamp, defaults to now

Routes: all under `/profiles`, mounted via a router in `routers/profiles.py`:

Method	Path	Status code	Behavior
`POST`	`/profiles`	201	Create a profile from `{email, display_name}`. Reject duplicate email with 409. Reject empty `display_name` with 400.
`GET`	`/profiles/{id}`	200	Return the profile. 404 if not found.
`DELETE`	`/profiles/{id}`	204	Delete the profile. 404 if not found. **Protected** — requires `X-Token: secret-token` (use the dependency from lesson 07). GET and POST are unprotected.

Output schema: the response should always include `id`, `email`, `display_name`, `created_at` (ISO 8601). Never the password (there isn't one — just a reminder that response models hide internal fields).

Suggested order of attack

1. Add the SQLAlchemy model to `models.py` (look at `Item` for the shape; use `DateTime` and `func.now()` for `created_at`).
2. Restart the api so `Base.metadata.create_all` picks up the new table. Verify with `\dt` in psql.
3. Create `routers/profiles.py`. Pattern after `routers/items.py` (after lesson 11). Two Pydantic models: `ProfileIn` and `ProfileOut`.
4. Mount the router in `main.py` with `app.include_router(profiles.router)`.
5. Apply `Depends(get_current_user)` only to the DELETE route, not to the router.
6. For the duplicate-email check, query the DB before inserting — if `db.query(Profile).filter(Profile.email == ...).first()` returns something, raise 409.
7. For the empty-`display_name` check, validate in the route handler (or use a Pydantic validator).
8. Test every path with curl and with Swagger.

The acceptance test (run when you think you're done)

These commands must all behave exactly as described. **Don't move on if any fail.**

1. POST a valid profile.

```
curl -i -X POST http://localhost:8000/profiles \
  -H "Content-Type: application/json" \
  -d '{"email":"a@example.com","display_name":"Alice"}'
```

Expected: `201 Created`. Response body has `id`, `email`, `display_name`, `created_at`.

2. POST the same email again.

```
curl -i -X POST http://localhost:8000/profiles \
  -H "Content-Type: application/json" \
  -d '{"email":"a@example.com","display_name":"Alice2"}'
```

Expected: `409 Conflict`.

3. POST with empty display_name.

```
curl -i -X POST http://localhost:8000/profiles \
  -H "Content-Type: application/json" \
  -d '{"email":"b@example.com","display_name":""}'
```

Expected: `400 Bad Request`.

4. GET the existing profile (use the id from step 1).

```
curl -i http://localhost:8000/profiles/1
```

Expected: `200 OK` with the profile.

5. GET a non-existent profile.

```
curl -i http://localhost:8000/profiles/999999
```

Expected: `404 Not Found`.

6. DELETE without the auth header.

```
curl -i -X DELETE http://localhost:8000/profiles/1
```

Expected: `422 Unprocessable Entity` (header missing — Pydantic-level rejection from the dependency).

7. DELETE with the wrong token.

```
curl -i -X DELETE http://localhost:8000/profiles/1 -H "X-Token: nope"
```

Expected: `401 Unauthorized`.

8. DELETE with the right token.

```
curl -i -X DELETE http://localhost:8000/profiles/1 -H "X-Token: secret-token"
```

Expected: `204 No Content`.

9. GET that deleted profile.

```
curl -i http://localhost:8000/profiles/1
```

Expected: `404 Not Found`.

10. Look at the database directly to confirm the table exists and the row is gone.

```
docker compose exec db psql -U postgres -d appdb -c "\d profiles"
docker compose exec db psql -U postgres -d appdb -c "SELECT * FROM profiles;"
```

Expected: schema printout, then a result that does not contain the deleted row.

Final submission

When all 10 acceptance steps pass:

1. Take a screenshot of your terminal showing the output of the full acceptance test (or at least steps 1, 2, 4, 5, 7, 8, 9).
2. Show your `/docs` page — Swagger should now display three sections: **items**, **users**, and **profiles**.
3. Submit the screenshot to the instructor.

Then:

```
echo "Module 03 final checkpoint: passed $(date -Iseconds)" >> ~/docker-course/notes/notes.txt
```

What you should be able to do now

You built a real backend. Reflect for a minute on what's in `~/docker-course/fastapi-app/`:

- A Python project with a venv, dependency-pinned via `requirements.txt`
- A FastAPI app with three resources, each in its own router
- Pydantic input + output models with type-driven validation
- A reusable auth dependency
- A Dockerfile that packages the whole thing
- A Compose stack with Postgres and a healthcheck
- SQLAlchemy ORM models persisted to a real database
- Proper HTTP status codes and error handling
- Auto-generated Swagger docs at `/docs`

That's the shape of every real Python backend service. The same patterns scale to thousands of routes.

What's next

Module 04 picks up from here — your FastAPI app becomes the codebase for AI-assisted refactoring, testing, and agent integration. Don't delete `~/docker-course/fastapi-app/`.

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.